

StayNest

A SUMMER INTERNSHIP REPORT

Submitted by:

RIYA PATEL

230640107011

In partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

In

COMPUTER ENGINEERING

**K. J. INSTITUTE OF ENGINEERING AND TECHNOLOGY, SAVLI,
VADODARA**



Gujarat Technological University, Ahmedabad

2026



K. J. INSTITUTE OF ENGINEERING AND TECHNOLOGY, SAVLI,
VADODARA

COMPUTER ENGINEERING

CERTIFICATE

This is to certify that the project entitled "StayNest" has been carried out by Patel Riya (230640107011). under my guidance in partial fulfillment for the degree of Bachelor of Engineering in COMPUTER ENGINEERING, 7th Semester of Gujarat Technological University, Ahmedabad during the academic year 2026-27.

Guides:

Mrs.Kalyani Deshpande
(Associate Professor)

Mr. AJAY BARIA
(Head of Department)

INTERNSHIP OFFER LETTER

Dear Patel Riya Arvindhbai,

We are pleased to offer you a 1 Month Internship as Project Intern at Codec Technologies, a global platform dedicated to empowering learners and connecting diverse talent to create meaningful career opportunities worldwide. Codec Technologies delivers dedicated IT and business consultancy services across 27+ countries, empowering global innovation and strategic growth.

Internship Details:

- Designation : MERN Stack Developer Intern
- Location & Mode: Pan India, Hybrid
- Duration : 06/07/2026 to 06/08/2026
- Reporting to : Assigned Project Head(s)

This program provides an immersive experience, enabling you to gain industry-level knowledge and acquire valuable skills. Exceptional interns may also qualify for a scholarship award based on weekly performance reviews conducted by project supervisor.

If you accept this offer and the terms above, please complete your assigned projects and training tasks on our platform and via email.
Best regards,

Sincerely,

Dr. Anurag Shrivastava,
Codec Technologies India
NCS ID: E19E86-0116588288923
Date Of Issue - 06/07/2026


Dr. Anurag Shrivastava
Talent Acquisition Manager



CERTIFICATE OF INTERNSHIP

This certificate is awarded to
Patel Riya

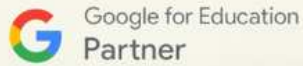
From Codec Technologies Pvt. Ltd.

In recognition of his/her efforts and achievements in completing the 1 Month AICTE & ICAC

Approved internship program as

MERN Stack Developer Intern

Conducted from 06/07/2026 to 06/08/2026



AICTE ID - CORPORATE6759d549ce59e1733940553

Dr. Anurag Shrivastava
Program Manager



K. J. INSTITUTE OF ENGINEERING AND TECHNOLOGY, SAVLI, VADODARA
COMPUTER ENGINEERING

DECLARATION

I hereby declare that the project entitled "StayNest" submitted in partial fulfillment for the degree of Bachelor of Engineering in COMPUTER ENGINEERING to Gujarat Technological University, Ahmedabad, is a Bonafede record of original project work carried out by me at "Codec Technologies Pvt. Ltd." Under the supervision of Mrs. Kalyani Deshpande and that no part of this report has been directly copied from any student's reports or taken from any other source, without providing due reference.

Name of the Student

Riya Patel
(230640107011)

Sign of Student.

ACKNOWLEDGEMENT

I would like to sincerely thank Codec Technologies Pvt. Ltd. for giving me the chance to complete my internship in Web Design with MERN Stack. This experience has been extremely valuable, helping me connect the gap between classroom knowledge and real world application knowledge.

I am especially thankful to who guided, me throughout the internship. His advice and feedback helped me understand full-stack development in the better way and motivated me to improve my work and skills.

I am also grateful to the Department of Computer Engineering, K. J. Institute of Engineering and Technology, and my faculty members for their continuous support and encouragement which played a important role in the successful completion of this project.

With sincere regards,
RIYA PATEL
(230640107011)

ABSTRACT

This report summarizes my internship at Codec Technologies Pvt. Ltd.”, where I worked in the domain of Web Design with MERN Stack. The objective of the internship was to gain practical experience in full-stack development by designing and implementing an e-commerce Web App.

The Project included features such as user authentication, product catalog, shopping cart, order management, and payment integration using Razorpay. The system was developed using React for the frontend, Node.js and Express for backend and mongoDB for database management along with bootstrap for responsive design.

This internship helped me to strengthen my technical knowledge of the MERN Stack and provided valuable exposure of industry practices in modern web application.

Beyond technical skills, the internship also taught me the importance of structured development, writing clean code which can be very necessary for me to learn to good enough for my future to work in any firm.

LIST OF FIGURES

No.	Description	Page
3.4	System Architecture diagram	6
3.5.1	Customer Use case	7
3.5.2	Admin Use case	8
3.6	User Journey	9
3.7	Sequence Diagram	10
3.8	Order process diagram	11

LIST OF TABLES

No.	Description	Page
4.1.1	USER TABLE	6
4.1.2	PROPERTY TABLE	7
4.1.3	BOOKING TABLE	8
4.1.4	REVIEW TABLE	9
4.1.5	LOCATION TABLE	10

TABLE OF CONTENTS

Serial No.	Description	Page No.
	ACKNOWLEDGEMENT	
	ABSTRACT	
	LIST OF FIGURES	
	LIST OF TABLES	
	TABLE OF CONTENTS	
1	CHAPTER – 1 INTRODUCTION 1.1 Project Summary 1.2 Project Details 1.3 Project Specification 1.4 Plan of Work 1.5 Materials/Tools Required	10
2	CHAPTER – 2 DESIGN 2.1 Design Methodology 2.2 Implementation Strategy	13
3	CHAPTER – 3 SYSTEM ANALYSIS 3.1 Study of Current System 3.2 Problem & Weakness of Current System 3.3 Feasibility Study 3.4 Function of System 3.5 Selection of Hardware & Software & Justification	16

4	CHAPTER – 4 SYSTEM DESIGN 4.1 Data Dictionary 4.2 System Architecture 4.3 ER Diagram 4.4 Data Flow Diagram	24
5	CHAPTER – 5 IMPLEMENTATION 5.1 User Authentication 5.2 User Dashboard 5.3 Property Listing & Search 5.4 Property Details 5.5 Booking Management 5.6 Payment Module 5.7 Wishlist & Reviews 5.8 Responsive UI Design 5.9 Testing & Debugging	28
6	CHAPTER – 6 PROTOTYPE 6.1 Signup Page 6.2 Login Page 6.3 Home Page 6.4 Create/AddPage 6.5 Review Pag 6.6 Edit Page 6.7 Location page	32
7	CHAPTER – 7 SUMMARY 7.1 Summary of Project	36
8	REFERENCES	37

CHAPTER 1

INTRODUCTION

1.1 Project Summary

StayNest – Online Property Rental & Booking Platform is a full-stack web application inspired by online property rental platforms. It allows users to search and view available properties, check property details, and make bookings online. Users can create accounts, manage their bookings, save properties to a wishlist, and provide reviews. Property owners can add and manage their property listings. The system provides a simple, responsive, and user-friendly interface for managing the complete property rental process.

Technologies Used: React.js, Node.js, Express.js, MongoDB, HTML, CSS and JavaScript.

1.2 Project Details

The StayNest application is designed with two main roles: **User and Property Owner**.

User Side:

- Account registration and login with authentication.
- Browsing and searching for available properties.
- Viewing property details, images, location and pricing.
- Booking properties for selected dates.
- Viewing and managing booking history.
- Adding and removing properties from the wishlist.
- Providing reviews and ratings for booked properties.
- Managing personal profile information.

Property Owner Side:

- Registering and managing owner profile.
- Adding new property listings.
- Editing and deleting property details.
- Uploading property images and information.
- Managing property availability.

- Viewing and managing customer bookings.
- Checking reviews and ratings received for properties.

All modules are integrated to provide a complete online property rental and booking solution.

1.3 Project Specification

The technical details of StayNest are as follows:

- **Frontend:** React.js for developing a responsive and interactive user interface.
- **Backend:** Node.js and Express.js for handling API requests, authentication and business logic.
- **Database:** MongoDB for storing user, property, booking, wishlist and review data.
- **Architecture:** MERN Stack with RESTful API communication between frontend and backend.
- **Authentication:** Secure user authentication and authorization for different user roles.
- **Development Tools:** Visual Studio Code (IDE), GitHub for version control and Postman for API testing.
- **Database Connectivity:** Mongoose for connecting the Node.js application with MongoDB.
- **Deployment:** The application can be deployed using suitable cloud hosting platforms.

1.4 Plan of Work

The work on StayNest was carried out step by step over four weeks:

1. **Week 1 – Requirement Analysis & Setup:** Understanding project requirements, designing the basic system structure, setting up the development environment and installing required tools and libraries.
2. **Week 2 – Frontend Development:** Creating responsive user interfaces using React components, implementing navigation and developing property listing, search and property detail pages.
3. **Week 3 – Backend Development:** Developing REST APIs using Node.js and Express.js, connecting the application with MongoDB and implementing user, property and booking management.
4. **Week 4 – Integration & Testing:** Connecting frontend with backend APIs, implementing authentication, wishlist, booking and review features, testing all modules and preparing the

application for deployment.

1.5 Materials/Tools Required

To complete StayNest, the following tools and resources were used:

- **Hardware:** Laptop with minimum 8 GB RAM, i3/i5 processor and stable internet connection.
- **Software:** Node.js, MongoDB, Visual Studio Code and Git.
- **Frameworks & Libraries:** React.js, Express.js, Mongoose and Node.js.
- **Database:** MongoDB for storing and managing application data.
- **Development Tools:** GitHub for version control and Postman for API testing.
- **Browser:** Google Chrome for testing and debugging the application.
- **Other Resources:** Internet connection and suitable deployment platform for hosting the application.

CHAPTER 2

DESIGN

2.1 Design Methodology

The design of **StayNest – Online Property Rental & Booking Platform** follows a structured and modular approach to provide a simple, secure, and user-friendly property rental experience. The application is developed using the **MERN stack**, which consists of MongoDB, Express.js, React.js, and Node.js. This architecture helps in developing a scalable web application with efficient communication between the frontend, backend, and database.

- ☐ **Frontend (React.js):** The user interface is developed using reusable React components. Different pages such as Home, Property Listing, Property Details, Login, Signup, Booking, Wishlist, User Dashboard, and Property Owner Dashboard are designed to provide smooth navigation and interaction. CSS is used to create a responsive interface suitable for desktops, tablets, and mobile devices.
- ☐ **Backend (Node.js + Express.js):** The backend is responsible for handling API requests, business logic, authentication, property management, booking operations, wishlist management, and review functionality. Express.js is used to create RESTful APIs that allow communication between the frontend and database.
- ☐ **Database (MongoDB):** MongoDB is used as the database for storing application information. It stores user details, property information, booking records, wishlist data, and reviews. Mongoose is used to define schemas and perform database operations.
- ☐ **Authentication & Authorization:** User authentication is implemented to provide secure access to the application. Different access levels are provided according to the user's role. Users can manage their bookings and wishlist, while property owners can add and manage their property listings.
- ☐ **Property Management:** Property owners can add property details such as property name,

location, description, price, images, and availability. They can also update or remove their listed properties.

- **Booking Management:** Users can select a property and make a booking based on the available dates. Booking information is stored in the database and can be viewed and managed through the user dashboard.
- **Architecture:** A client-server architecture is followed in which the React frontend communicates with the Node.js and Express.js backend through REST APIs. The backend processes requests and communicates with MongoDB for storing and retrieving data.

This modular design allows different components of StayNest to be developed, tested, and maintained independently. It also makes the application easier to extend with additional features in the future.

2.2 Implementation Strategy

The implementation of **StayNest** was carried out in different phases to ensure proper development, integration, and testing of all modules.

1. Frontend Development:

- Created the user interface using React.js components.
- Developed pages for home, property listings, property details, login, signup, booking, wishlist, and dashboards.
- Implemented navigation and routing between different pages.
- Designed a responsive layout for different screen sizes.

2. Backend API Development:

- Developed RESTful APIs using Node.js and Express.js.
- Implemented APIs for user registration and login.
- Created APIs for adding, viewing, updating, and deleting property listings.
- Implemented booking, wishlist, and review-related operations.
- Added role-based access for users and property owners.

3. Database Setup:

- Created MongoDB collections for users, properties, bookings, wishlists, and reviews.
- Used Mongoose for schema definition and database operations.
- Established relationships between users, properties, and bookings.
- Implemented data validation to maintain data consistency.

4. Integration Phase:

- Connected the React frontend with the Express.js backend APIs.
- Connected the backend with MongoDB.
- Integrated authentication with the frontend and backend.
- Implemented property search, booking, wishlist, and review functionality.
- Connected user and property owner dashboards with the required backend services.

5. Testing & Refinement:

- Tested individual modules such as authentication, property management, booking, wishlist, and reviews.
- Performed integration testing to verify communication between frontend, backend, and database.
- Tested different user roles and application workflows.
- Identified and resolved errors and improved the overall user interface and application performance.

The phased implementation strategy helped in developing StayNest systematically and ensured that all major modules worked together as a complete property rental and booking platform.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Study of Current System

The current system is an online property rental and booking platform named **StayNest**, which is designed to manage users, property listings, bookings, wishlists, and reviews. The system consists of a React-based frontend, Node.js and Express.js backend, and MongoDB for database management. The platform allows users to search for properties, view property details, check availability, and make bookings online. Property owners have the ability to add, update, and manage their property listings.

Key features of the current system include:

- **User Management:** Users can register, log in, and manage their account information.
- **Property Management:** Property owners can add, edit, and delete their property listings.
- **Property Search:** Users can search and browse available properties according to their requirements.
- **Booking Management:** Users can select available dates and make property bookings.
- **Wishlist:** Users can save properties to their wishlist for future reference.
- **Review Management:** Users can provide reviews and ratings for properties after their stay.
- **Owner Dashboard:** Property owners can manage their properties and view booking information.

3.2 Problem & Weakness of Current System

The current system, while functional, has some areas that can be improved to provide a better user experience and operational efficiency:

1. Limited Search and Filtering Options:

- The system provides basic property searching, but advanced filters based on price range, location, property type, and amenities can improve the property discovery experience.

2. Booking Availability Management:

- Property availability needs to be properly maintained to avoid conflicts between different bookings. A more advanced availability management system can improve booking accuracy.

3. Limited Booking Notifications:

- Users may not receive detailed notifications for booking confirmation, cancellation, or updates. Adding notification functionality can improve communication between users and property owners.

4. Property Verification:

- The system may not include an advanced property verification mechanism. Adding verification features can help maintain the quality and reliability of property listings.

5. Review Management:

- Reviews and ratings can be improved by providing better review management and filtering options. This can help users make informed decisions while selecting a property.

3.3 Feasibility Study

Objective of Feasibility Study:

The objective of this study is to determine whether the StayNest property rental and booking system is feasible from **operational, technical, economic, and schedule** perspectives. The study also helps identify whether the proposed system can be developed and maintained using the available resources.

Operational Feasibility:

The StayNest system is operationally feasible because it supports important property rental operations such as user registration, property listing, property searching, booking management, wishlist management, and reviews. The system provides separate functionality for users and property owners, making it easier to manage different activities. With proper testing and maintenance, the system can provide a smooth property rental experience.

Technical Feasibility:

The system is developed using **React.js, Node.js, Express.js, and MongoDB**, which are widely used technologies for modern web application development. The system is technically feasible because these technologies support responsive interfaces, RESTful APIs, database management, authentication, and scalable application development.

Additional features such as advanced search, notifications, online payment, and map integration can also be implemented using suitable APIs and services.

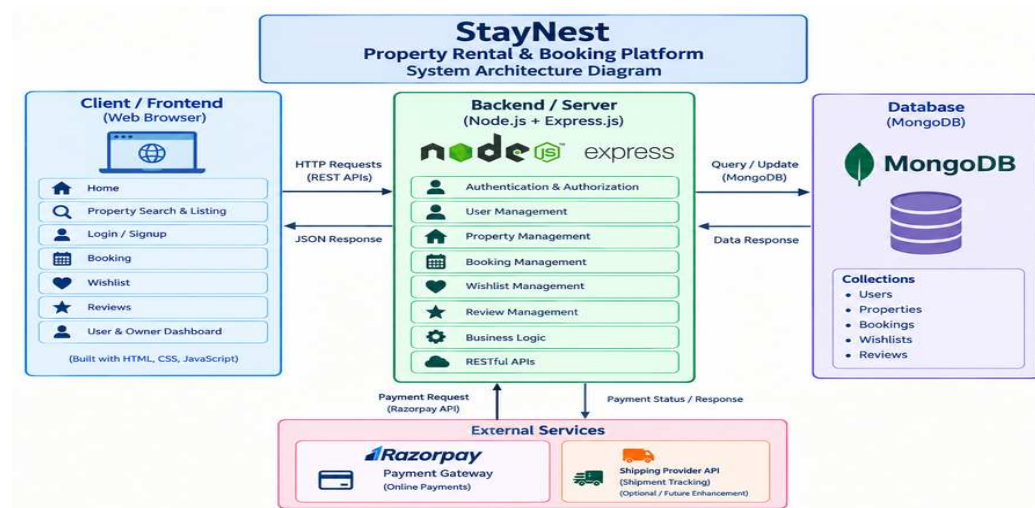
Economic Feasibility:

The StayNest system is economically feasible because it uses open-source technologies such as React.js, Node.js, Express.js, and MongoDB, which reduces software licensing costs. Development can be performed using commonly available hardware and software tools such as Visual Studio Code and GitHub. Additional costs may be required for hosting, domain registration, payment gateway services, or third-party APIs depending on the deployment requirements.

Schedule Feasibility:

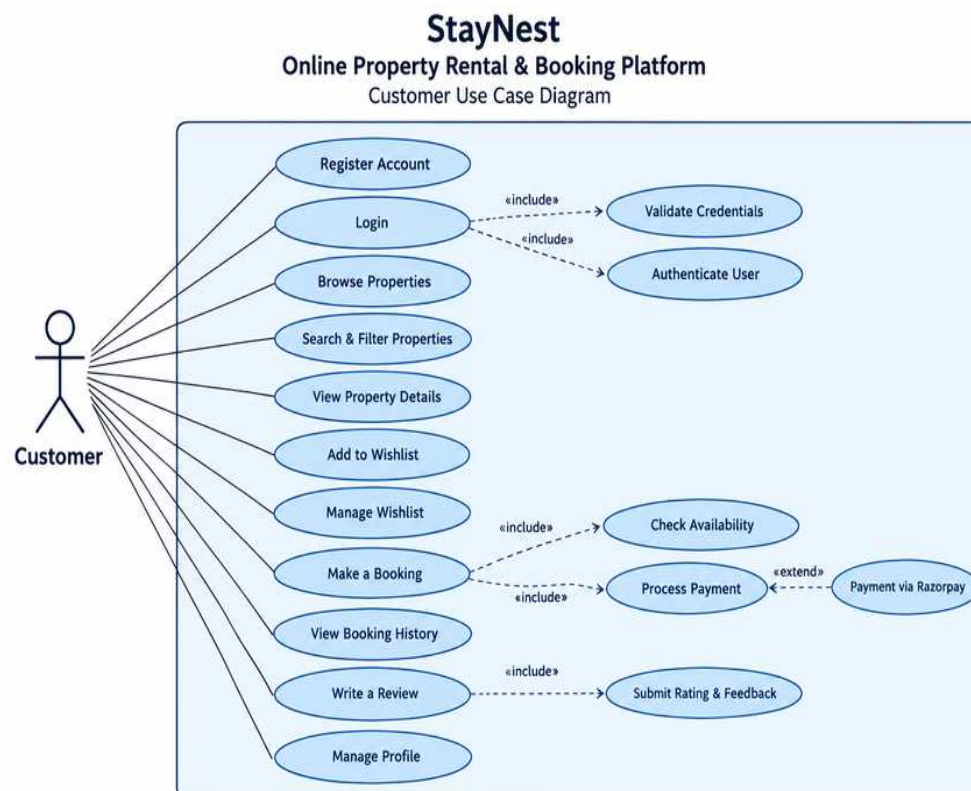
The project is schedule-feasible because development can be divided into different phases such as requirement analysis, frontend development, backend development, database integration, testing, and deployment. Proper planning and implementation of each module within the planned development period allows the project to be completed systematically.

3.4 System Architecture Diagram:

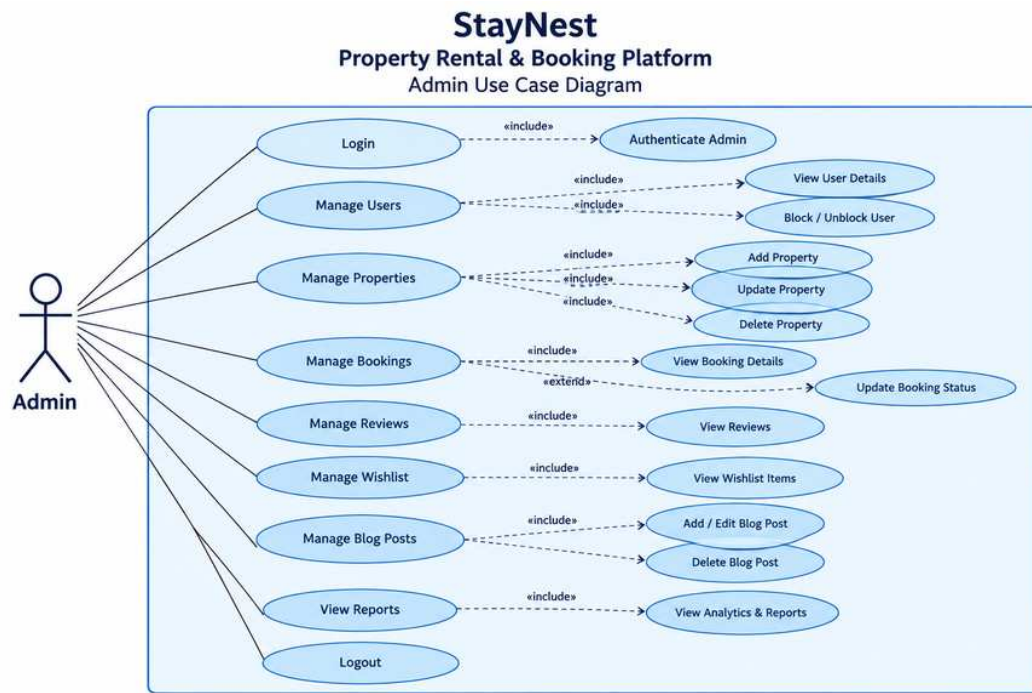


3.5 Functions of System:

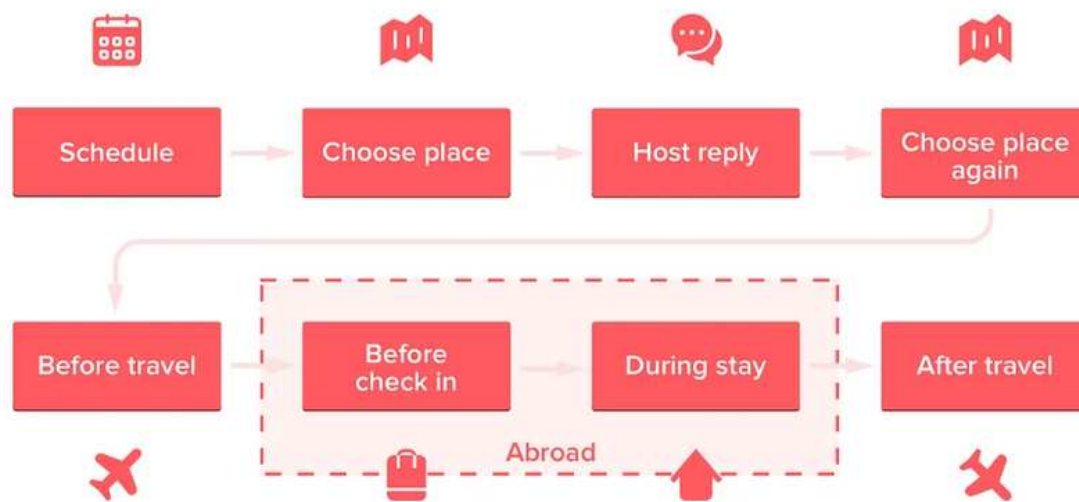
3.5.1 Customer Use Case:



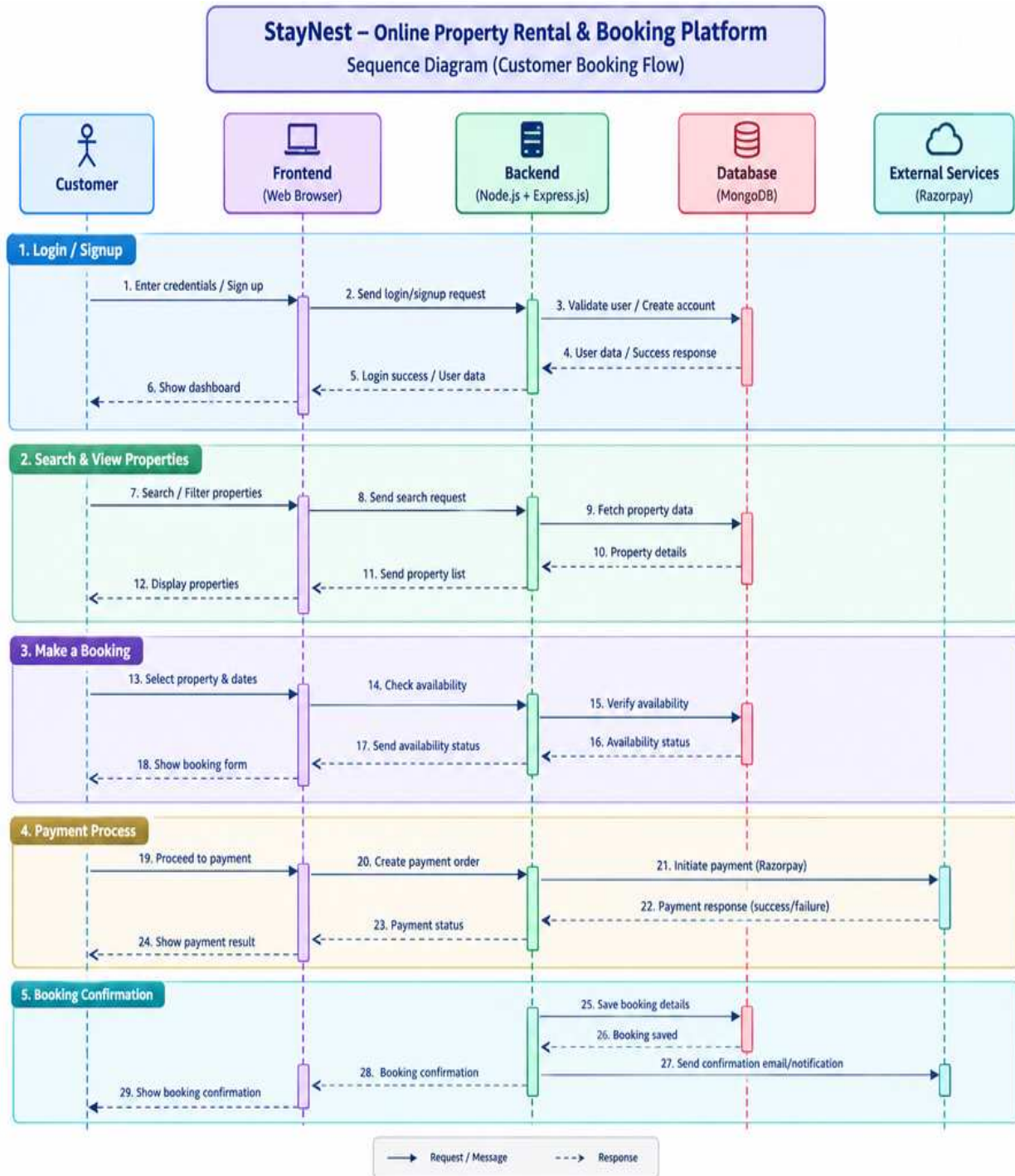
3.5.2 Admin Use Case:



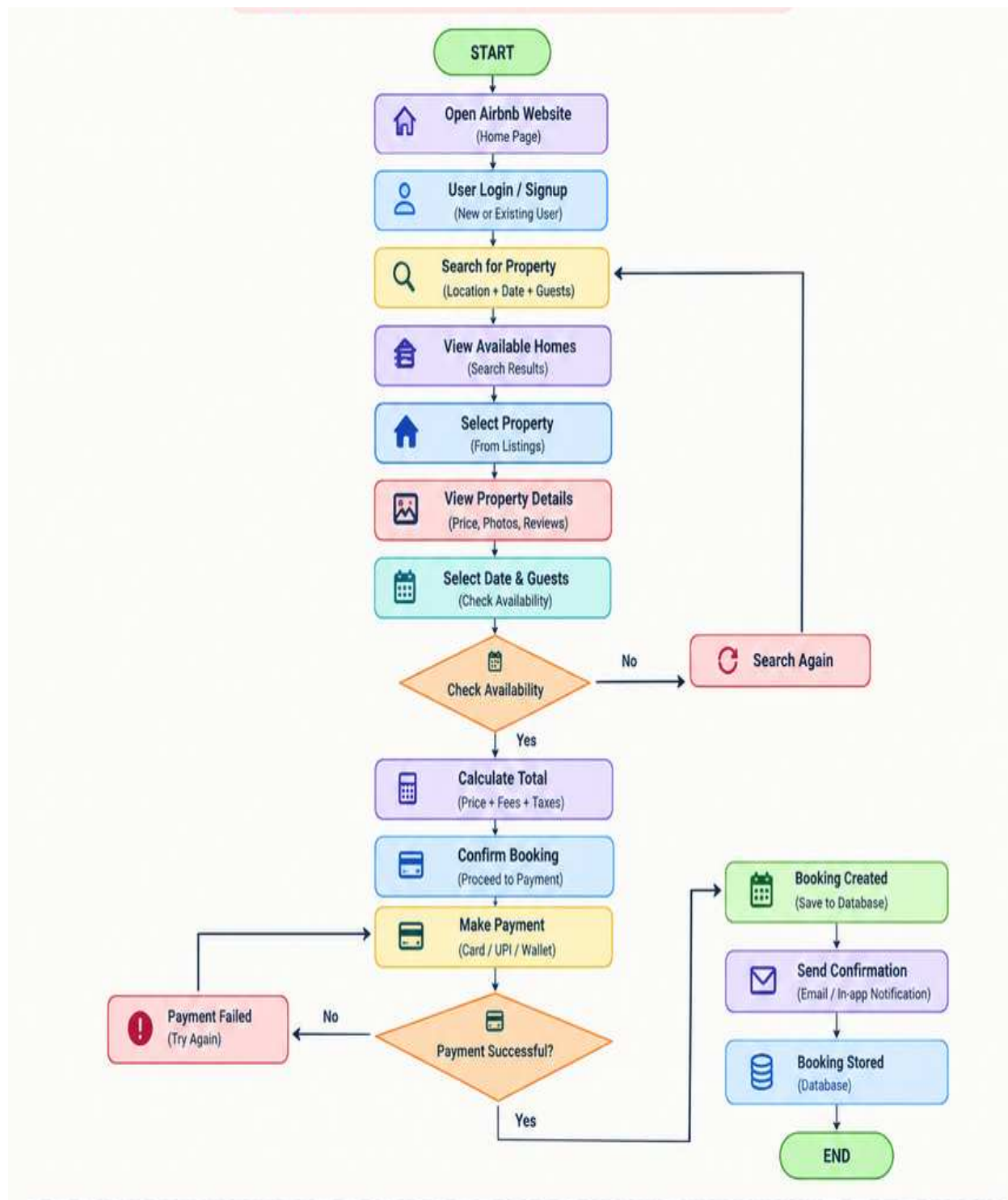
3.6 User Journey:



3.7 Sequence Diagram:



3.8 Order Process Diagram:



3.9 Selection of Hardware & Software & Justification:

3.9.1 Hardware:

RAM	4 GB
DEVICE	WINDOWS/PC

3.9.2 Software:

OPERATING SYSTEM	WINDOWS/MAC/LINUX
WEB BROWSER	GOOGLE, CHROME
APPLICATION SOFTWARE	REQUIRED PACKAGE TO BE INSTALLED
DATABASE	MONGODB
PAYMENT INTEGRATION	PAYMENT INTEGRATION
VS STUDIO	API TESTING
VERSION CONTROL	PACKAGE MANAGER

CHAPTER 4

SYSTEM DESIGN

4.1 Data Dictionary

The data dictionary describes the main data entities used in the Airbnb Clone application. The system uses MongoDB as the database, where data is stored in collections.

4.1.1 User Collection

Sr. No.	Field Name	Data Type	Size	Constraint	Key	Description
1	user_id	ObjectId	24	Not Null	PK	Unique identifier for each user
2	name	String	50	Not Null		Contains user's full name
3	email	String	100	Not Null, Unique		User's email address
4	password	String	255	Not Null		Contains hashed password
5	role	String	20	Not Null		User role such as Guest, Host or Admin
6	phone	String	15	Optional		User's contact number
7	address	String	200	Optional		User's address
8	createdAt	Date		Not Null		Date and time when the account was created

4.1.2 Property / Listing Collection

Sr. No.	Field Name	Data Type	Size	Constraint	Key	Description
1	property_id	ObjectId	24	Not Null	PK	Unique identifier for each property
2	host_id	ObjectId	24	Not Null	FK	References the user who owns the property
3	title	String	100	Not Null		Name/title of the property
4	description	String	1000	Not Null		Detailed description of the property
5	location	String	200	Not Null		Location of the property
6	price	Number		Not Null		Price per night
7	guests	Number		Not Null		Maximum number of guests allowed
8	bedrooms	Number		Not Null		Number of bedrooms
9	images	Array		Not Null		Images of the property
10	amenities	Array		Optional		Facilities available at the property
11	createdAt	Date		Not Null		Date and time when the property was added

4.1.3 Booking Collection

Sr. No.	Field Name	Data Type	Size	Constraint	Key	Description
1	booking_id	ObjectId	24	Not Null	PK	Unique identifier for each booking
2	user_id	ObjectId	24	Not Null	FK	References the guest who made the booking
3	property_id	ObjectId	24	Not Null	FK	References the booked property
4	checkIn	Date		Not Null		Check-in date
5	checkOut	Date		Not Null		Check-out date

Sr. No.	Field Name	Data Type	Size	Constraint	Key	Description
6	guests	Number		Not Null		Number of guests
7	totalAmount	Number		Not Null		Total booking amount
8	status	String	20	Not Null		Booking status such as Pending, Confirmed, Cancelled or Completed
9	createdAt	Date		Not Null		Date and time when booking was created

4.1.4 Payment Collection

Sr. No.	Field Name	Data Type	Size	Constraint	Key	Description
1	payment_id	ObjectId	24	Not Null	PK	Unique identifier for each payment
2	booking_id	ObjectId	24	Not Null	FK	References the booking associated with the payment
3	user_id	ObjectId	24	Not Null	FK	References the user who made the payment
4	amount	Number		Not Null		Amount paid for the booking
5	payment_status	String	20	Not Null		Payment status such as Success, Failed or Refunded
6	payment_gateway	String	50	Not Null		Payment gateway used, such as Razorpay
7	transaction_id	String	100	Not Null		Unique transaction/payment identifier
8	payment_date	Date		Not Null		Date and time when payment was made

4.1.5 Review Collection

Sr. No.	Field Name	Data Type	Size	Constraint	Key	Description
1	review_id	ObjectId	24	Not Null	PK	Unique identifier for each review
2	user_id	ObjectId	24	Not Null	FK	References the user who posted the review
3	property_id	ObjectId	24	Not Null	FK	References the reviewed property
4	rating	Number		Not Null		Rating given to the property
5	comment	String	500	Optional		User's review/comment

Sr. No.	Field Name	Data Type	Size	Constraint	Key	Description
6	createdAt	Date		Not Null		Date and time when the review was posted

4.1.6 Wishlist Collection

Sr. No.	Field Name	Data Type	Size	Constraint	Key	Description
1	wishlist_id	ObjectId	24	Not Null	PK	Unique identifier for the wishlist
2	user_id	ObjectId	24	Not Null	FK	References the user who owns the wishlist
3	property_id	ObjectId	24	Not Null	FK	References the saved property
4	createdAt	Date		Not Null		Date and time when the property was added to wishlist

4.1.7 Relationship Between Collections

The major relationships in the Airbnb Clone system are:

- **User → Property:** A host can create multiple properties.
- **User → Booking:** A user can make multiple bookings.
- **Property → Booking:** A property can have multiple bookings.
- **Booking → Payment:** A booking is associated with its payment.
- **User → Review:** A user can write multiple reviews.
- **Property → Review:** A property can receive multiple reviews.
- **User → Wishlist:** A user can save multiple properties in the wishlist.

CHAPTER 5

IMPLEMENTATION

The implementation of **StayNest** was carried out in a systematic manner to develop an efficient and user-friendly hotel and room booking system. The development process included frontend development, backend API development, database integration, user authentication, room management, booking management, and payment functionality. Each module was developed and tested individually before integrating it into the complete system.

5.1 User Registration and Authentication

StayNest provides a secure registration and login system for users. New users can create an account by providing the required information such as name, email address, and password.

User authentication is implemented using **JWT (JSON Web Token)** to provide secure access to protected features. Passwords are securely hashed before being stored in the database. After successful login, an authenticated user can access booking and other account-related functionalities.

5.2 User Dashboard

The user dashboard provides a personalized interface for registered users. Users can access and manage their account information through the dashboard.

The dashboard allows users to:

- View their profile information.
- View their previous bookings.
- Check current and upcoming bookings.
- View booking details and status.
- Manage their account information.

This module provides users with easy access to their booking-related information.

5.3 Hotel and Room Listing

StayNest provides a dedicated section for displaying available hotels and rooms. Each listing contains important information such as hotel name, room type, location, room images, description, price, and available facilities.

The room and hotel information is stored in the database and retrieved dynamically through backend APIs. This allows the available accommodation information to be displayed and updated efficiently.

5.4 Search and Filter

A search and filtering mechanism is implemented to help users find suitable accommodation easily.

Users can search for hotels or rooms based on relevant information such as:

- Location
- Room type
- Price range
- Availability
- Other available facilities

The search and filter functionality reduces the time required to find suitable accommodation and improves the overall user experience.

5.5 Room Booking

The booking module allows users to reserve an available room by selecting the required check-in and check-out dates.

Before confirming the booking, the system verifies the availability of the selected room. The user provides the required booking information and confirms the reservation.

After successful booking, the booking details are stored in the database along with information such as user, room, booking dates, and booking status.

5.6 Booking and Payment

StayNest provides a checkout and payment process for completing room reservations. The user can review the selected room, booking dates, and total amount before

confirming the booking.

An online payment gateway can be integrated to process secure payments. After successful payment confirmation, the booking information is stored in the database and the booking status is updated accordingly.

The payment and booking information helps maintain a proper record of completed reservations.

5.7 Booking Management

The booking management module allows users to monitor their reservations. Users can view details of their current and previous bookings.

The system maintains different booking statuses such as:

- Pending
- Confirmed
- Completed
- Cancelled

This allows users and administrators to easily track the current state of each reservation.

5.8 Admin Dashboard

StayNest provides an admin dashboard for managing the overall hotel and booking system. The administrator has access to various management functionalities.

The admin can:

- Add new hotels or rooms.
- Update room and hotel information.
- Delete unavailable or outdated listings.
- View registered users.
- View customer bookings.
- Manage booking status.
- Monitor room availability.

The admin dashboard helps in maintaining accommodation information and managing reservations efficiently.

5.9 Database Integration

The database is used to store and manage important information required by the

StayNest system. Data related to users, hotels, rooms, bookings, and other system entities is stored in the database.

Backend APIs are used to perform operations such as inserting, retrieving, updating, and deleting records. Database integration ensures that the information remains organized and can be accessed whenever required.

5.10 Responsive User Interface

The StayNest interface is designed to provide a simple and responsive user experience. The frontend is developed so that users can access the application from different devices, including desktops, tablets, and smartphones.

The interface contains user-friendly pages for registration, login, hotel and room browsing, room details, booking, and user dashboard. Responsive design ensures that the application remains accessible across different screen sizes.

5.11 Testing and Debugging

Testing was performed at different stages of development to ensure that the StayNest application works correctly and reliably.

Unit Testing

Individual modules such as user authentication, room listing, search, booking, and database operations were tested separately to verify their functionality.

Integration Testing

Integration testing was performed to ensure proper communication between the frontend, backend APIs, database, authentication system, and payment module.

End-to-End Testing

Complete workflows were tested from the user's perspective. The testing process included user registration, login, searching for rooms, checking availability, making a booking, completing payment, and viewing booking details.

Debugging

Errors identified during development and testing were resolved through iterative debugging. The application was repeatedly tested after fixing issues to ensure that the changes did not affect other modules.

CHAPTER – 6

PROTOTYPE

6.1 SignUp page

Signup on wanderlust

Username

Email

Password

SignUp

6.2 Login Page

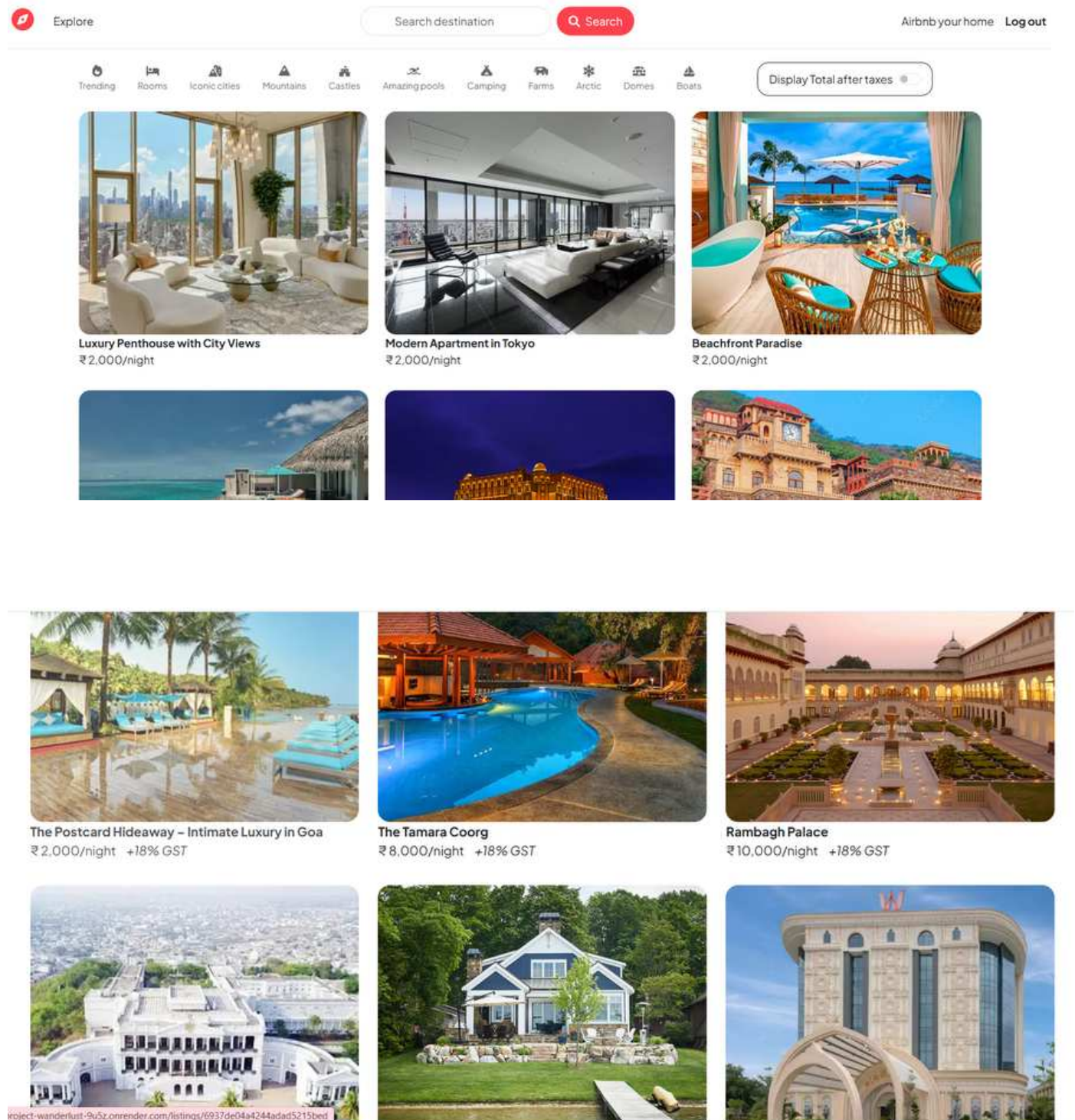
Login

Username

Password

Login

6.3 Home Page & Listings



6.4 Create/Add listings

CREATE A NEW LISTING

Title

Description

Upload Image

No file chosen

Price

Country

Location

6.5 Review page

Leave a Review

Rating



Comment

All Reviews

@riya9

Amazing place it is !!

6.6 Edit page

Edit Your Listing

Title

Luxury Penthouse with City Views

Description

Indulge in luxury living with panoramic city views from this stunning penthouse apartment.

original listing image



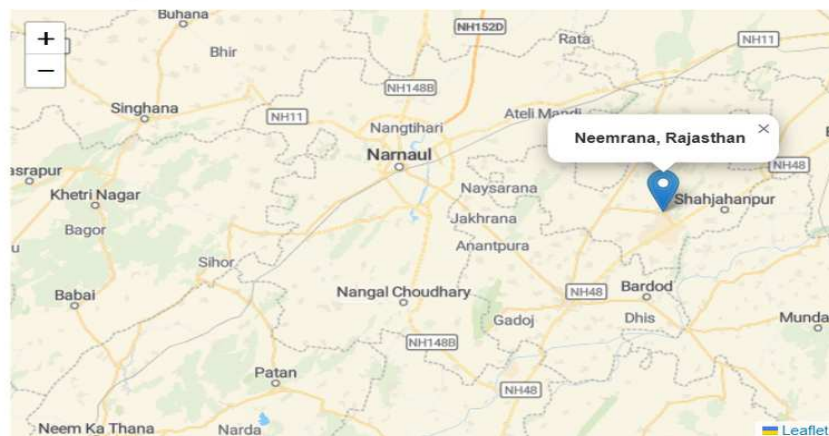
Upload new Image

Choose file

No file chosen

6.7 location

Where you'll be



CHAPTER 7

PROJECT SUMMARY

StayNest is a web-based hotel and room booking system designed to provide users with a simple and convenient platform for finding and booking suitable accommodation. The system allows users to register and log in securely, search for available hotels and rooms, view room details, check availability, and make bookings.

The application provides a user dashboard where users can manage their profile and view their current and previous bookings. An admin dashboard is also provided to manage hotels, rooms, users, and booking information. The system maintains booking details and their status, making it easier for both users and administrators to manage reservations.

StayNest uses a database to store and manage important information such as user details, hotel and room details, and booking records. Backend APIs are used to handle communication between the frontend and database. The system also supports a secure payment process for completing online reservations.

The main objective of StayNest is to simplify the hotel booking process by providing an organized, responsive, and user-friendly platform. It reduces the need for manual booking processes and provides users with quick access to accommodation information and reservation details.

REFERENCES

1. React.js Documentation – <https://react.dev>
2. Node.js Official Documentation – <https://nodejs.org>
3. Express.js Documentation – <https://expressjs.com>
4. MongoDB Documentation – <https://www.mongodb.com/docs>
5. Bootstrap Official Documentation – <https://getbootstrap.com>
6. Razorpay API Documentation – <https://razorpay.com/docs>
7. Visual Studio Code – <https://code.visualstudio.com>
8. Postman API Testing Tool – <https://www.postman.com>